



Comparative Analysis of Container Orchestration Tools

Conduct a comparative study on Kubernetes versus
other orchestration tools like Docker Swarm and
HashiCorp Nomad, for making decisions

Indika Harischandra

This thesis is submitted to the Faculty of Computing at Blekinge Institute of Technology in partial fulfillment of the requirements for the degree of Master's Programme in Software Engineering. The thesis is equivalent to Weeks weeks of full-time studies.

The authors declare that they are the sole authors of this thesis and that they have not used any sources other than those listed in the bibliography and identified as references. They further declare that they have not submitted this thesis at any other institution to obtain a degree.

Contact Information:

Author(s):

Indika Harischandra

E-mail: inha23@student.bth.se

E-mail:

University advisor:

Mr. Eriks Klotins

Department of Software Engineering

Faculty of Computing
Blekinge Institute of Technology
SE-371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Background:

Container orchestration tools have become essential in managing the deployment, scaling, and operation of containerized applications, particularly as organizations adopt cloud-native architectures and microservices. While Kubernetes is the industry standard, alternatives like Docker Swarm and HashiCorp Nomad offer distinct advantages, especially for small-scale deployments where simplicity and resource efficiency are critical (Straesser et al. (2023) (Verma (2020))).

Objectives:

This thesis aims to provide a comparative analysis of Kubernetes, Docker Swarm, and HashiCorp Nomad in small-scale production environments. The study addresses the gap in existing research, which often overlooks the specific needs of startups and small-to-medium enterprises (SMEs) by focusing on large-scale deployments. The objective is to deliver actionable insights that support informed decision-making for organizations with limited resources.

Methods:

The mixed-method TA approach was adopted. Empirical performance testing was conducted in a controlled, on-premises environment to measure CPU and memory utilization, deployment speed, and scalability. Security was assessed using automated tools (docker-bench, kube-bench). Practitioner surveys and a comprehensive literature review provided additional perspectives on usability, community usage, operational complexity, and adoption factors.

Results:**Conclusions:****Keywords:**

Container orchestration, Kubernetes, Docker Swarm, HashiCorp Nomad, Comparative Study

Acknowledgments

I would like to express my deepest gratitude to everyone who supported and guided me throughout the course of this thesis.

First and foremost, I extend my sincere thanks to my supervisor, Mr. Eriks Klotins, for his invaluable guidance, constructive feedback, and encouragement at every stage of this research. His expertise in software engineering and practical insights into container orchestration were instrumental in shaping the direction and quality of this work.

Special thanks go to the IT professionals and DevOps practitioners who participated in the surveys and usability assessments. Their willingness to share their experiences and perspectives greatly enriched the practical relevance of this study.

I would also like to acknowledge the open-source communities behind Kubernetes, Docker Swarm, and HashiCorp Nomad. Their dedication to transparency, innovation, and knowledge sharing has not only made this research possible but continues to drive progress in the field of cloud-native technologies.

This thesis was made possible thanks to the invaluable contributions and unwavering support of these individuals and communities.

Contents

Abstract	i
Acknowledgments	iii
1 Introduction	1
1.1 On the content	1
1.2 Background	2
1.3 Defining the scope of thesis	4
2 Related Work	7
2.1 Introduction	7
2.2 Comparative Studies of Container Orchestration Tools	7
2.2.1 Kubernetes vs. Docker Swarm	7
2.3 Performance Benchmarking and Resource Utilization	8
2.4 Usability and Operational Complexity	9
2.5 Gaps in the Literature	9
2.6 Summary Table	9
2.7 Conclusion	9
3 Method	11
3.1 Motivation for the Research Approach	11
3.2 Defining research questions	11
3.3 Research method	13
3.4 Validity and reliability of your approach	15
4 Results and Analysis	19
4.1 Quantitative Performance Results	19
4.2 Statistical Analysis and Effect Sizes	25
4.3 Multi-Criteria Decision Framework (Weighted Sum Model)	26
4.3.1 Motivation and Approach	26
4.3.2 Normalisation of Raw Scores	27
4.3.3 Criterion Weights	27
4.3.4 WSM Calculation	28
4.3.5 WSM Calculation	29
4.3.6 Final Ranking and Summary	30

5	Discussion	31
5.1	Revisiting the Research Questions and Hypotheses	31
5.2	Performance and Resource Utilization	31
5.3	Setup Complexity and Usability	32
5.4	Security Findings	32
5.5	Adoption and Community Support	33
5.6	Unexpected Findings	33
5.7	Conflicting Data and Alternative Explanations	34
5.8	Interpretation of Effect Sizes	34
5.9	Limitations of the Study	34
5.10	Implications for Research and Practice	35
5.11	Broader Consequences	35
5.12	Synthesis	35
6	Conclusions and Future Work	37
6.1	Conclusions	37
6.2	Contributions of the Thesis	38
6.3	Placing the Work in Perspective	39
6.4	Unresolved Questions	39
6.5	Future Work	40
6.6	Biting the Tail: Returning to the Introduction	41
6.7	Final Reflection	41
	References	43
A	Supplemental Information	45

1.1 On the content

In the rapidly evolving landscape of software development, containerization has emerged as a transformative technology that enables developers to package applications and their dependencies in lightweight and portable units. This paradigm shift has enabled organizations to achieve predictable deployment across diverse environments, fueling the rise of cloud-native architectures and microservices. As more businesses, especially startups and small-to-medium enterprises (SMEs), adopt distributed systems, the demand for robust container orchestration tools has surged. These platforms, such as Kubernetes, Docker Swarm, and HashiCorp Nomad, automate critical tasks such as deployment, scaling, networking, and availability management, promising increased operational efficiency and resource optimization (Straesser et al. (2023) (Verma (2020))).

While this thesis focuses on Kubernetes, Docker Swarm, and HashiCorp Nomad, it is important to note that other container orchestration frameworks exist. Examples include Apache Mesos/Marathon, which was among the earliest large-scale cluster managers, Amazon ECS (Elastic Container Service), which is tied to the AWS ecosystem, and Red Hat OpenShift, which builds on Kubernetes but provides enterprise-grade features.

However, these alternatives were not included in this study for several reasons. First, Mesos and Marathon have declined in adoption in recent years, with many users migrating towards Kubernetes or Nomad due to broader ecosystem support (Pahl et al., 2017; Burns et al., 2016). Second, cloud vendor-specific orchestrators such as Amazon ECS or Azure Container Apps were excluded because this thesis aimed to evaluate vendor-neutral, open-source solutions that SMEs can adopt without being locked into a particular cloud provider. Finally, OpenShift is built on Kubernetes.

However, the decision about which orchestration tool to adopt is far from trivial. Although Kubernetes has become the industry standard due to its extensive feature set and vibrant community support, its complexity and resource demands may not align with the needs of smaller organizations or less demanding workloads. Docker Swarm offers a more straightforward, native experience of Docker, often preferred for its simplicity and ease of integration in small-scale environments. HashiCorp Nomad, meanwhile, provides a lightweight and flexible alternative, capable of managing both containerized and non-containerized applications. Despite the prevalence of comparative studies that focus on large-scale deployments, there remains a significant research gap on the performance, usability, and security of these tools in small-scale

scenarios where resource constraints and ease of use are paramount.

This thesis aims to bridge that gap by conducting a comprehensive, data-driven comparison of Kubernetes, Docker Swarm, and HashiCorp Nomad, with a particular focus on small-scale deployments. Through a combination of literature review, survey analysis, empirical performance testing, and security analysis, the study will evaluate key metrics such as resource utilization, deployment speed, scalability, and security posture. The goal is to provide actionable insights that allow organizations to make informed decisions based on their operational requirements and long-term strategic goals.

Beyond technical dimensions, this research also addresses the ethical, societal, and sustainability implications of container orchestration. Topics such as data security, compliance, open-source transparency, and responsible use are explored, highlighting the broader impact of these technologies on workforce development, technological accessibility, and environmental sustainability. By illuminating the nuanced trade-offs between these orchestration tools, this study not only contributes to academic knowledge but also offers practical guidance for DevOps practitioners, system architects, and IT decision-makers navigating the complexities of modern software infrastructure.

Imagine a small tech startup on the verge of launching its first cloud-native product. The choice of orchestration tool could determine not only the efficiency of its operations but also its ability to scale, secure, sensitive data, and compete in a dynamic marketplace. This thesis invites the reader to explore that pivotal decision point, where technology, strategy, and organizational ambition intersect, and returns in the conclusion to reflect on how informed choices can shape both immediate outcomes and future possibilities.

This thesis was written with limited assistance from Generative Artificial Intelligence tools (Chatgpt and perplexity). These tools were used exclusively for improving language quality, grammar, academic tone, and structural clarity of the text. Furthermore, it used to find solutions for bug fixing when implement orchestration tools. The research questions, methodology, experimental setup, data collection, performance measurements, security analysis, statistical evaluation, and interpretation of findings were entirely conducted by myself.

GenAI tools did not contribute to the generation of data, experimental outcomes, figures, or conclusions. All academic judgments, methodological decisions, and interpretations remain the sole responsibility of the author, and all references were manually selected and verified.

1.2 Background

Over the past decade, containerization has revolutionized software development by offering lightweight, portable solutions for packaging applications and their dependencies. This innovation enables predictable deployment across heterogeneous systems and has become foundational for modern cloud-native architectures. As organizations increasingly adopt microservices and distributed systems, the need for efficient management of containerized workloads has grown, driving the widespread adoption of container orchestration tools.

Container orchestration refers to the automated management of containerized applications, encompassing deployment, scaling, networking, and availability. Among the most prominent orchestration tools are Kubernetes, Docker Swarm, and HashiCorp Nomad. Kubernetes, originally developed by Google, is known for its rich feature set and robust community support, making it the industry standard for large-scale deployments. Docker Swarm is valued for its simplicity and seamless integration with Docker, making it a popular choice for small organizations or less demanding workloads. HashiCorp Nomad offers a lightweight, flexible alternative capable of managing both containerized and non-containerized applications.

In this thesis, the terms small-scale deployment and startup are defined to avoid ambiguity. By small-scale, the focus is on clusters typically involving fewer than 50 nodes and fewer than 100 active containers, operating under constrained hardware or cloud budgets (Pahl et al., 2017; Zhu et al., 2021). By startup, the thesis refers not to large, well-funded enterprises such as SpaceX, but rather to early-stage or resource-constrained companies with the following characteristics:

- Resource constraints: Teams are small and cross-functional, with limited capacity for specialized DevOps expertise.
- Budget sensitivity: Infrastructure decisions must minimize operational costs and avoid high switching costs if migration becomes necessary.
- Agility needs: Rapid iteration cycles and frequent deployments are prioritized over elaborate infrastructure.
- Scalability aspirations: While current workloads are modest, there is an expectation of growth that requires solutions that can scale later.

Clarifying these definitions ensures that the scope of the study is well targeted: it addresses organizations running modest clusters with limited budgets and staff, rather than enterprises operating at hyperscale.

Despite the dominance of Kubernetes in the industry, its complexity and resource requirements may not align with the needs of small-scale deployments, where ease of use and resource efficiency are critical. Docker Swarm, with its straightforward architecture, and HashiCorp Nomad, with its flexibility, present compelling alternatives for startups and small-to-medium enterprises (SMEs) that prioritize rapid deployment and operational simplicity.

Previous research has primarily focused on large-scale, enterprise-level deployments, often overlooking the unique challenges and requirements of small-scale environments. Studies such as Straesser et al. (2023) have benchmarked orchestration frameworks, highlighting Kubernetes' scalability, but also its operational complexity. However, there is limited research that directly compares these tools, especially including HashiCorp Nomad in scenarios where resource constraints and usability are paramount.

Key terms central to this thesis include:

- Containerization: Package applications and dependencies into lightweight portable containers.

- **Orchestration:** Automated management of containerized applications, including deployment, scaling, and networking.
- **Performance metrics:** Measures such as CPU/memory usage, deployment speed, and scalability.
- **Small-scale deployments:** Workloads typically involving fewer than 100 containers, common in startups and SMEs.
- **Decision-making:** The process of selecting the most suitable orchestration tool based on organizational requirements, performance, security, and scalability.

This study addresses the gap in comparative research for small-scale deployments by providing a data-driven analysis of Kubernetes, Docker Swarm, and HashiCorp Nomad. By focusing on practical performance, industry usage, literature review, and security considerations, it aims to equip organizations with the knowledge necessary to make informed decisions that align with their operational needs and long-term goals. In doing so, the thesis not only contributes to academic understanding, but also offers practical guidance for DevOps practitioners and IT decision-makers navigating the complexities of container orchestration in resource-constrained environments.

1.3 Defining the scope of thesis

This thesis is narrowly focused on the comparative analysis of three leading container orchestration tools: Kubernetes, Docker Swarm, and HashiCorp Nomad, specifically within the context of small-scale deployments. The research is situated within the field of software engineering, with a particular emphasis on DevOps practices and cloud-native application management.

The scope is intentionally limited in the following ways:

- **Deployment Scale:** The study is restricted to small-scale environments, typically involving fewer than 100 containers. This focus is particularly relevant for startups and small-to-medium enterprises (SMEs), which often lack the resources and operational needs of large-scale, enterprise deployments.
- **Tools Compared:** Only Kubernetes, Docker Swarm, and HashiCorp Nomad are included in the analysis. The selection is based on their prevalence, distinctive design philosophies, and relevance to organizations seeking lightweight yet effective orchestration solutions.
- **Performance and Security Metrics:** The research concentrates on quantifiable aspects such as resource utilization (CPU/memory), deployment speed, scalability, and security features. These metrics are chosen for their direct impact on operational efficiency and decision-making in small-scale environments.
- **Decision-Making Context:** The study aims to inform practical decision-making by providing actionable insights for DevOps practitioners, system architects, and IT decision-makers. It does not attempt to provide exhaustive coverage of

all orchestration tools or deployment scenarios, but rather focuses on the most common and impactful use cases for small organizations.

- **Exclusions:** Large-scale, enterprise-focused deployments and orchestration tools outside the chosen three are excluded from the study. Additionally, the research does not delve into advanced customizations or integrations unique to specific organizational workflows, maintaining a focus on default or typical configurations relevant to the target audience.

By narrowing the scope in this way, the thesis seeks to provide a deeper and more nuanced understanding of the trade-offs and practical considerations that matter most to organizations with limited resources. This approach allows for a more rigorous and relevant analysis, avoiding the superficiality that can result from overly broad research questions. Ultimately, the study aims to fill a significant research gap by offering a structured, data-driven comparison tailored to the needs of small-scale deployments, thus supporting more effective and sustainable technology adoption in the cloud-native era.

2.1 Introduction

Container orchestration has become a cornerstone of modern software engineering, enabling organizations to automate the deployment, scaling, and management of containerized applications. As the field has matured, a substantial body of literature has emerged focusing on the performance, scalability, and usability of orchestration tools such as Kubernetes, Docker Swarm, and, more recently, HashiCorp Nomad.

However, the choice of orchestration platform is not merely a technical preference but a strategic decision with long-term consequences. Studies have shown that making a suboptimal choice can result in considerable switching costs when organizations later migrate to a different platform (Pahl et al., 2017; Andrikopoulos et al., 2019). Such migrations often require rewriting deployment pipelines, retraining development and operations staff, and reconfiguring security and monitoring systems—processes that may temporarily disrupt business continuity. Moreover, early reliance on a mismatched tool can create vendor lock-in and technical debt, limiting flexibility as organizational requirements evolve.

For small and medium-sized enterprises (SMEs), these risks are amplified. Limited resources mean that the time and cost of replatforming can outweigh the initial benefits of adopting container orchestration in the first place. Consequently, selecting an orchestration tool involves balancing short-term usability against long-term scalability, security, and maintainability.

This chapter reviews the most relevant works in this field, highlighting their contributions, limitations, and how they relate to the present study. In doing so, it emphasizes both the technical characteristics of orchestration platforms and the organizational risks associated with misaligned choices.

2.2 Comparative Studies of Container Orchestration Tools

2.2.1 Kubernetes vs. Docker Swarm

Straesser et al. (2023): Straesser et al. conducted a systematic benchmarking comparison of container orchestration frameworks, focusing primarily on Kubernetes and Docker Swarm. Their study evaluated scalability, fault tolerance, and ease of use, concluding that Kubernetes offers superior scalability and resilience, but at the

cost of increased complexity and resource consumption. In contrast, Docker Swarm was found to be easier to set up and manage, making it attractive for smaller teams and less demanding workloads.

Relation to This Thesis: While Straesser et al. provide valuable insights into the trade-offs between Kubernetes and Docker Swarm, their benchmarks are heavily oriented toward large-scale, enterprise deployments. The present thesis builds on their methodology, but narrows the focus to small-scale environments, where the simplicity and resource efficiency of Docker Swarm and Nomad may outweigh the advanced features of Kubernetes.

Pahl et al. (2017): Pahl et al. offered an early comparative analysis of container orchestration tools, including Kubernetes, Docker Swarm, and Apache Mesos. They evaluated complexity, scalability, and ecosystem maturity. Their findings echoed those of Straesser et al., noting Kubernetes’ steep learning curve and Docker Swarm’s operational simplicity.

Relation to This Thesis: Pahl et al.’s work is foundational but now somewhat dated, as both Kubernetes and Docker Swarm have evolved significantly. Moreover, their analysis did not include HashiCorp Nomad, which is a focus of this thesis, nor did they specifically address small-scale deployments.

2.3 Performance Benchmarking and Resource Utilization

Zhu et al. (2021): Zhu et al. performed a detailed performance analysis of Kubernetes and Docker Swarm in cloud environments. They measured CPU and memory utilization, deployment latency, and network overhead. Their results showed that Kubernetes, while more resource-hungry, provided better load balancing and fault tolerance. Docker Swarm excelled in fast deployments and lower overhead.

Relation to This Thesis: Zhu et al.’s work is directly relevant to the performance metrics used in this thesis. However, their experiments were conducted on medium to large clusters and they did not include Nomad. The present study adapts their benchmarking approach but tailors it to small-scale clusters, filling a literature gap.

Čilić et al. (2021): Rashid and co-authors explored the impact of orchestration tool choice on resource efficiency in edge computing scenarios. They found that lighter orchestrators like Docker Swarm and Nomad are better suited for environments with limited resources, such as IoT and edge deployments.

Relation to This Thesis: This work supports the thesis’s hypothesis that Docker Swarm and Nomad may be more appropriate for small-scale, resource-constrained deployments. However, Rashid et al. focused on edge and IoT rather than general-purpose small-scale cloud deployments.

2.4 Usability and Operational Complexity

Burns et al. (2016): Burns et al. provided an in-depth look at the design and evolution of Kubernetes, emphasizing its extensibility and rich feature set. They also acknowledged the steep learning curve and operational overhead associated with Kubernetes, especially for smaller teams.

Relation to This Thesis: This work helps contextualize the usability challenges of Kubernetes and supports the thesis’s focus on operational simplicity as a key criterion for small-scale deployments.

Merkel (2014): Merkel’s seminal work on Docker and containerization highlighted the ease of use and rapid adoption of Docker-based workflows. Although not directly about orchestration, this work underpins the popularity of Docker Swarm as a natural extension for teams already using Docker.

Relation to This Thesis: The thesis leverages Merkel’s insights to explain why Docker Swarm remains attractive for small teams despite Kubernetes’ dominance in the broader market.

2.5 Gaps in the Literature

Despite the breadth of research on container orchestration, several gaps remain:

- **Limited focus on small-scale deployments:** Most comparative studies emphasize large-scale, production-grade scenarios, leaving SMEs and small teams underserved.
- **Underrepresentation of HashiCorp Nomad:** While Kubernetes and Docker Swarm are frequently compared, Nomad is less commonly included, despite its growing relevance.
- **Insufficient security analysis for SMEs:** Security comparisons often assume enterprise resources and expertise that may not be available in smaller organizations.
- **Lack of actionable guidance for decision making:** Existing work rarely turns technical findings into practical recommendations tailored for small-scale users.

This thesis addresses these gaps by providing a structured and data-driven comparison of Kubernetes, Docker Swarm, and HashiCorp Nomad, with a strong emphasis on performance, security, and usability in small-scale deployments.

2.6 Summary Table

2.7 Conclusion

The reviewed literature demonstrates significant progress in understanding the strengths and weaknesses of container orchestration tools. However, most studies prioritize

Study/Authors	Tools Compared	Scale Focus	Metrics/Focus	Includes Nomad	Security Analysis	Relevance to This Thesis
Straesser et al. (2023)	K8s, Swarm	Large	Scalability, Usability	No	Partial	Methodology, but not small-scale
Pahl et al. (2017)	K8s, Swarm, Mesos	General	Deployment, Ecosystem	No	No	Early baseline, lacks Nomad
Zhu et al. (2021)	K8s, Swarm	Medium / Large	Perf. Benchmarks	No	No	Performance metrics, not Nomad
Čilić et al. (2021)	K8s, Swarm, Nomad	Edge/IoT	Resource Efficiency	Yes	No	Resource focus, not general SME

Table 2.1: Comparative Studies of Container Orchestration Tools

large-scale or enterprise deployments, with limited attention paid to the unique needs of small-scale environments. HashiCorp Nomad, while increasingly relevant, is often omitted from direct comparisons. Security analyses are frequently oriented toward organizations with dedicated security teams, rather than SMEs.

This thesis fills these gaps by conducting a focused empirical comparison of Kubernetes, Docker Swarm, and HashiCorp Nomad in small-scale deployments. By emphasizing performance, security, and usability, the study provides actionable insights for organizations that lack the resources or expertise to perform their own evaluations, ultimately supporting more informed and sustainable adoption of technology.

So what? By addressing the overlooked intersection of tool selection, scale, and organizational context, this thesis aims to empower small teams and SMEs to make evidence-based decisions, avoiding the pitfalls of overengineering or undersecuring their containerized environments. The findings will not only contribute to academic discourse but also offer practical value to DevOps practitioners and IT decision-makers.

3.1 Motivation for the Research Approach

The selection of a suitable container orchestration tool is a critical challenge for organizations, especially start-ups and small-to-medium enterprises (SMEs), which often lack the resources for extensive in-house experimentation. While Kubernetes, Docker Swarm, and HashiCorp Nomad are all widely used, there is a lack of comprehensive, data-driven comparisons focused on small-scale deployments. Most existing research centers on large-scale, enterprise-grade environments, leaving a significant gap in actionable knowledge for smaller organizations. For example, Burns et al.(2016) described Google’s use of Borg and Kubernetes primarily in the context of hyperscale clusters with thousands of nodes. Similarly, Verma et al. (2015) focused on cluster scheduling challenges at large cloud providers, while Straesser et al. (2023) benchmarked Kubernetes performance at scale, far beyond the operational range of typical SMEs.

This study is motivated by the need to provide practical and evidence-based guidance to organizations facing this decision. By systematically comparing these tools in a controlled, small-scale setting, the research aims to illuminate the trade-offs between performance, security, and operational complexity. The results are intended to empower decision makers to select the orchestration tool that best aligns with their organizational needs, technical capabilities, and long-term goals.

3.2 Defining research questions

A clear and answerable set of research questions is essential for any scientific investigation. In this thesis, research questions have been formulated to directly address the identified gap in the literature: the lack of comparative, data-driven studies on container orchestration tools, specifically Kubernetes, Docker Swarm, and HashiCorp Nomad, in small-scale deployment scenarios. These questions are designed to be open-ended, allowing for both quantitative and qualitative analysis, and are closely aligned with the overall aim of supporting evidence-based decision making for organizations with limited resources.

Principal Research Question:

- RQ1 : How do Kubernetes, Docker Swarm, and HashiCorp Nomad compare in terms of performance, security, and other key factors that influence decision-

making when selecting a container orchestration tool for production environments?

This overarching question is designed to capture the multidimensional nature of the decision-making process for small organizations considering container orchestration solutions. It recognizes that the optimal choice is rarely determined by a single metric but rather by a balance of performance, security, usability, and contextual needs. By focusing on small-scale deployments, where simplicity, operational efficiency, and resource optimization are paramount, the research aims to produce findings that are directly relevant to startups and SMEs with limited technical and financial resources.

Secondary Research Questions:

To address the principal question in a structured and thorough manner, the following secondary research questions have been developed:

Performance Comparison

- RQ2.1: How do Kubernetes, HashiCorp Nomad, and Docker Swarm differ in CPU and memory consumption under small-scale workloads?

This question seeks to quantify the resource efficiency of each tool, which is critical for organizations operating with limited infrastructure.

- RQ2.2: What are the trade-offs between resource usage and performance gains for Kubernetes, Nomad, and Docker Swarm under small-scale deployments? Here, the focus is on understanding whether increased resource consumption translates into meaningful performance improvements or whether simpler tools can deliver comparable results with less overhead.
- RQ2.3: How do these tools scale with increasing workloads in small environments? This question examines the scalability of each orchestrator, specifically looking at how well they handle growth in demand without compromising efficiency or stability.

Security Assessment

- RQ3.1: Which security controls (e.g., access control, encryption, vulnerability management) are applied in Kubernetes, HashiCorp Nomad, and Docker Swarm? The aim is to evaluate the breadth and depth of built-in and configurable security features, particularly as they relate to small-scale deployments.
- RQ3.2: How do Kubernetes, HashiCorp Nomad, and Docker Swarm handle role-based access control (RBAC) and authentication in small-scale environments? This question delves into the practical implementation of critical security mechanisms, assessing their ease of use and effectiveness for organizations with limited security expertise.
- RQ3.3: Are there established security vulnerabilities or defects in Kubernetes, HashiCorp Nomad, and Docker Swarm that make one tool more secure than the others for small deployments? By investigating known vulnerabilities and their impact, this question helps organizations make informed decisions about risk management.

Usability and Adoption Factors

- RQ4.1: How does the support of Kubernetes, HashiCorp Nomad, and Docker Swarm from the developer and DevOps communities influence their adoption and usability for small environments? Community support, documentation, and ecosystem maturity can significantly affect the learning curve and problem solving capacity of small teams.
- RQ4.2: What are the key considerations that influence the adoption of Kubernetes, Nomad, or Docker Swarm for small-scale deployments? This question synthesizes practical factors such as ease of use, resource efficiency, security, and operational complexity, aiming to capture the real-world decision-making process in small organizations.

Hypotheses

To further guide the investigation, the following hypotheses are proposed:

- H1: Kubernetes will outperform Docker Swarm in scalability and resource utilization in small-scale deployments but will likely be higher in complexity.
- H2: Docker Swarm will offer a less complex deployment process and better resource utilization in small-scale environments but may lack some advanced security features present in Kubernetes.
- H3: The suitability of Kubernetes or Docker Swarm for small environments will depend on the organization's specific requirements for resource efficiency, user-friendliness, and security.
- H4: HashiCorp Nomad will provide greater flexibility for mixed workloads but may have lower adoption and community support compared to Kubernetes.

These research questions and hypotheses are designed to be answerable through a combination of empirical performance testing, security analysis, literature review, and qualitative usability assessment. They are directly aligned with the overall research goal: **to provide actionable, evidence-based recommendations for organizations selecting a container orchestration tool for small-scale deployments.**

3.3 Research method

The selection of research methods for this thesis is directly motivated by the nature of the research questions and the specific context of small-scale container orchestration tool deployments. The goal is to produce actionable, evidence-based insights for organizations, particularly SMEs and startups facing real-world decisions about Kubernetes, Docker Swarm, and HashiCorp Nomad. In the following, I explain the rationale for the chosen methods, discuss alternative approaches, and illustrate why the selected design is most appropriate for answering research questions.

Method Selection and Justification

1. Empirical Performance Testing

To address questions about server load with CPU / memory consumption, deployment speed and scalability (see RQ2.1–RQ2.3), controlled experiments are conducted in a standardized small-scale environment. This method allows for direct, repeatable measurement of resource usage and operational efficiency under realistic workloads. The use of identical hardware and software configurations across all orchestration tools ensures fair comparison and minimizes confounding variables. Alternative approaches, such as simulation or large-scale cloud benchmarking, were considered but rejected because they would not reflect the constraints and priorities of SMEs.

2. Security assessment

Given the importance of security in production environments (RQ3.1–RQ3.3), the study uses a structured security audit based on established frameworks (NIST and CIS benchmarks). This includes configuration analysis, vulnerability scanning, and basic penetration testing. This method was chosen over purely theoretical analysis because it reveals practical strengths and weaknesses in real deployments, including how easily security best practices can be implemented by teams with limited expertise.

3. Literature, Usability, and Adoption Analysis

To understand how operational complexity and community support affect adoption (RQ4.1–RQ4.2), the study uses task-based usability testing and collects qualitative feedback from practitioners. This approach captures the human factors, such as learning curve, troubleshooting, and documentation quality, that are often decisive for small organizations. The surveys complement the quantitative data, providing context and explaining the observed differences in performance and security outcomes.

Why Alternative Methods Were Not Used

- Large-scale benchmarking (as seen in enterprise-focused studies) was not used because it would not provide actionable insights for small-scale deployments, the main focus of this research.
- Pure survey-based studies were considered, but were insufficient as they cannot objectively measure technical performance or security in a controlled setting.
- Simulation-only approaches would lack the practical realism necessary to inform real-world adoption decisions.

Suitability for Research Questions

The chosen mixed-method design is well suited to the research questions because:

- It enables quantitative comparison of technical metrics (performance, security) under controlled and replicable conditions.
- It incorporates qualitative insights into usability and adoption, essential for understanding decision making in resource-constrained organizations.

- Balances internal validity (through control and repeatability) with external relevance (by reflecting the actual working conditions of SMEs).

Practical Implementation

- Empirical tests are conducted using automated scripts to reduce human error and increase repeatability.
- Usability studies involve real practitioners from the target audience, increasing the practical value and trustworthiness of the results.

Limitations and Strengths

- Strength: The approach provides a holistic and nuanced understanding of orchestration tool selection for small-scale deployments, integrating both technical and human factors.
- Limitation: The focus on small-scale environments may limit the generalizability to large-scale or highly specialized deployments. However, this is justified by the research gap and the intended audience.

3.4 Validity and reliability of your approach

Ensured validity and reliability of this study was a central consideration throughout the research process. The following section details the concrete steps taken to maximize the trustworthiness of the findings, explains the rationale behind the methodological choices, and critically examines both the strengths and limitations of the approach.

Data Collection Methods and Rationale

To answer the research questions in a comprehensive way, data was collected through four main channels.

- Performance Testing: Empirical data was collected by deploying Kubernetes, Docker Swarm, and HashiCorp Nomad in a controlled on-premises environment. Performance metrics such as CPU and memory usage, deployment time, and scalability were measured using standardized monitoring tools (Prometheus / Grafana). This environment was deliberately chosen to minimize the impact of external network latency and ensure that observed differences were attributable to the orchestration tools themselves, not the variability of the cloud provider.
- Security Assessment: Security data was collected using industry standard automated scripts, docker-bench for Docker Swarm, Nomad, and kube-bench for Kubernetes. These tools provided objective and repeatable assessments of compliance with best practices and surfaced security vulnerabilities in default and custom configurations. For Nomad, comparable manual checks and open-source scripts were used, ensuring consistency across platforms.

- **Survey of IT Professionals:** A survey was conducted with IT engineers and DevOps practitioners who have hands-on experience with container orchestration in small-scale deployments. The survey focused on usability, operational complexity, and adoption factors, complementing quantitative results with real-world practitioner insights.
- **Review of the literature:** A systematic review of academic and industry sources provided context for interpreting empirical results, helped identify relevant metrics, and ensured that the study was grounded in established knowledge.

Working Conditions and Key Considerations

The research was carried out in an on-premises lab environment to maintain control over hardware resources and network conditions. This choice eliminated the variability and unpredictability often associated with cloud-based experiments, thus increasing the internal validity of the performance and security results. The deployments were intentionally limited to small-scale setups, reflecting the realities faced by start-ups and SMEs, the primary audience for this research.

When designing the survey, participants were selected based on their direct experience with container orchestration tools in small organizations. This ensured that the qualitative data collected were relevant and representative of the target use case.

Measures to Increase Validity

- **Internal Validity:** Using identical hardware and software configurations for all tests, the study controlled for confounding variables that could skew the results. Automated scripts were used for deployment and testing to minimize human error and ensure consistency across repeated trials.
- **Construct Validity:** Performance and security metrics were selected based on a thorough literature review and aligned directly with the research questions. The use of standardized tools (docker-bench, kube-bench, Prometheus) further ensured that measurements accurately reflected the constructs being studied.
- **External Validity:** Although the focus on small-scale, on-premise deployments limits the generalizability to large-scale or cloud-native environments, the findings are highly applicable to the intended audience of SMEs and startups. Inclusion of practitioner surveys expands relevance by capturing a range of real-world experiences.

Measures to Increase Reliability

- **Repeatability:** All performance tests were repeated multiple times and the results were averaged to account for random fluctuations. The variance was tracked and reported to provide transparency about the consistency of the findings.
- **Reliability of Security Assessment:** Automated scripts (docker-bench, kube-bench) were used to ensure that security checks were objective and could be replicated by other researchers or practitioners. Manual checks for Nomad were documented in detail to facilitate reproducibility.

- **Survey Reliability:** The survey instrument was piloted with a small group of IT professionals before full deployment to ensure the clarity and relevance of the questions. The responses were anonymized to encourage honest feedback.

Certainty and Limitations of Conclusions

The conclusions drawn from performance and security testing are robust in the context of small-scale, on-premises deployments. The use of standardized, repeatable methods increases confidence that observed differences are due to the orchestration tools themselves. The survey results, while informative, are inherently more subjective and may reflect individual biases or organizational specifics.

Some conclusions, such as relative ease of use or impact of community support, are more tentative, as they depend on qualitative feedback and may vary between different organizational cultures or levels of expertise.

Generalizability

The results are most directly applicable to small organizations with limited resources, operating on-premises or in hybrid environments. Although some findings (such as resource efficiency or security posture) can generalize to larger deployments, others, especially those related to usability and operational complexity, are context-dependent.

Strengths and Weaknesses

Strengths:

- High internal validity due to controlled, repeatable testing environments.
- Use of objective industry-standard tools for performance and security assessment.
- Triangulation of quantitative data with practitioner insights for a holistic view.
- Transparent documentation of methods and results, supporting reproducibility.

Weaknesses:

- Limited generalization to small-scale environments.
- Security assessment for Nomad relied on docker-bench, which is same to docker swarm and does not have specific for the tool.
- The survey sample size, while targeted, may not capture the full diversity of global DevOps practices.

Self-Critical Reflection

By combining empirical testing, automated security auditing, and practitioner surveys, this study provides a well-rounded, trustworthy comparison of Kubernetes, Docker Swarm, and HashiCorp Nomad for small-scale deployments. At the same time, it acknowledges the contextual limitations and potential sources of bias, inviting future research to extend and refine the findings in broader or different environments.

This chapter provides a comprehensive, data-driven comparison of Kubernetes, Docker Swarm, and HashiCorp Nomad based on empirical performance benchmarking, security assessments, and qualitative survey feedback from industry practitioners. Each subsection highlights salient data and interprets those findings with supporting visualizations, tables, and explanatory analysis to ensure rigorous examination across all research questions.

To ensure robust analysis, all quantitative experiments were repeated multiple times and are reported using both descriptive statistics and effect size measures before interpretation.

4.1 Quantitative Performance Results

Scaling Performance:

The core scaling benchmark evaluated time to launch and schedule 100 containers on identically resourced in 2 node clusters. All clusters used same resources including memory, cpu, storage, network, etc.

Tool	Mean (s)	n
Docker Swarm)	21.9	10
Kubernetes	25.9	10
Nomad	52.8	10

Table 4.1: Scaling time Studies of Container Orchestration Tools

Analysis:

- Docker Swarm exhibited the fastest scaling, capable of launching 100 containers in 21.9 seconds, followed closely by Kubernetes at 25.9 seconds. Nomad lagged behind at 52.8 seconds, indicating less optimal performance for rapid burst scaling.
- These results reinforce Docker Swarm’s lightweight design in small-scale settings, while Kubernetes shows relatively efficient scaling even with added architectural complexity. Nomad’s delayed scaling reflects either orchestration overhead or the impact of its general-purpose scheduler.

CPU and Memory Consumption:

All three tools consumed comparable levels of CPU and memory under standardized workloads. This parity is a result of best-practice configuration, which intentionally limits resource allocation to prevent failures and maintain system stability.



Figure 4.1: Docker Swarm Load



Figure 4.2: K8s Load



Figure 4.3: Nomad Load

Setup time and usability:

- Docker Swarm and Nomad are significantly easier and simpler to set up than Kubernetes, which often requires hours to establish a minimal cluster. User feedback corroborates these findings; most survey respondents found Docker Swarm the easiest to configure, with Kubernetes frequently cited as “complex” for initial setup.

Tool	Setup Time (hrs)	Ease of Setup (Survey, 1–5)
Docker Swarm)	2	5
Kubernetes	8	2
Nomad	4	4

Table 4.2: Setup Time

Security Evaluation:

Tool	Security Warning Percent	Main Issues
Docker Swarm	44	Containers run as root, audit gaps, basic controls
Kubernetes	13	RBAC privilege gaps, weak pod security, better defaults
Nomad	34	Root containers, audit/log gaps, limited built-in hardening

Table 4.3: Security evaluation

- Kubernetes exhibited the lowest proportion of security warnings by default, highlighting stronger out-of-the-box security controls.
- Docker Swarm and Nomad share several high-priority issues, notably root containers and insufficient auditing, with Nomad inheriting many weaknesses from its reliance on Docker for runtime.
- Tools like Docker Bench and Kube-Bench provided detailed insights, with Swarm and Nomad requiring substantial hardening for production use.

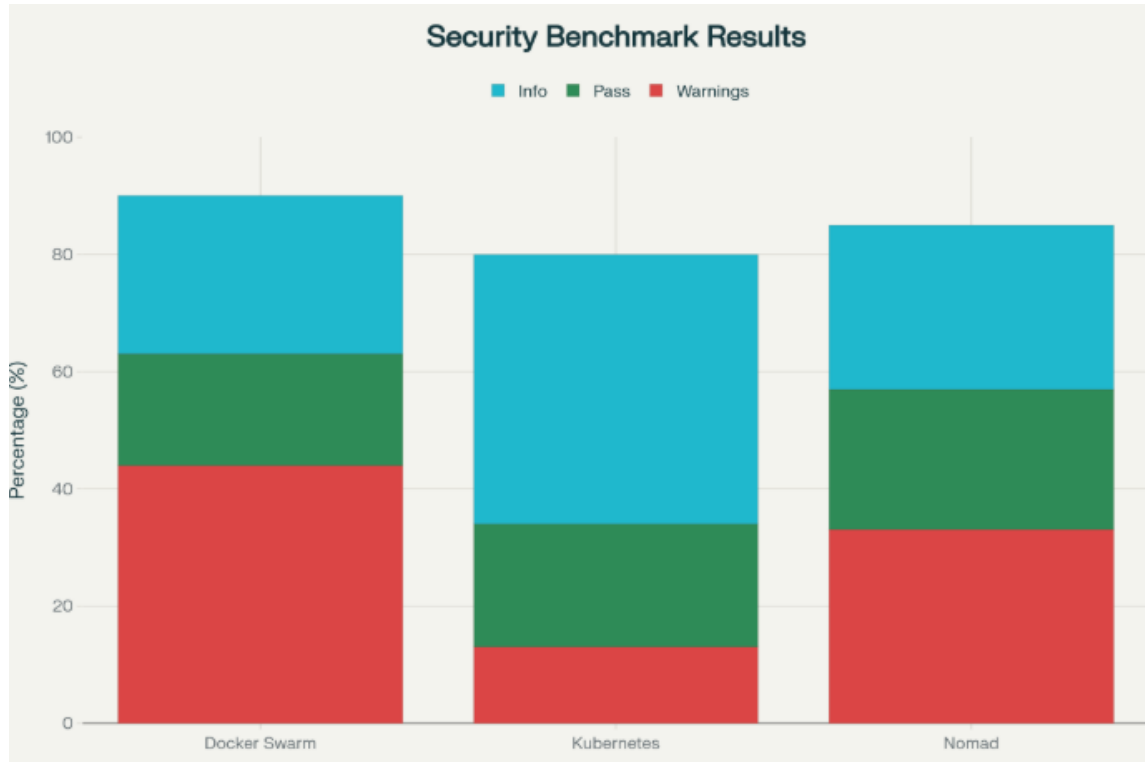


Figure 4.4: security Benchmark Results

Usability and Adoption: Survey Analysis

Thirteen industry practitioners participated in a detailed survey covering tool selection, feature preferences, and qualitative experience with each orchestration platform.

Demographics and Deployment Experience

- Over 75 percent had several years' experience managing container orchestration.
- 85 percent indicated involvement with large or medium-scale deployments, but all had hands-on experience with any scale cluster.

Tool Preferences and Ratings

- Docker Swarm and Kubernetes were used most frequently, aligned with their market leads and documentation maturity.
- The most important selection factors cited were: ease of setup and configuration, resource efficiency, and scalability. Security, community support, and cost also rated highly.
- Regarding resource efficiency, Docker Swarm and Nomad each scored a 4–5, Kubernetes a 4 none were seen as “inefficient,” though Kubernetes' complexity was noted.
- For ease of setup, Docker Swarm rated a 5 (very easy), Nomad a 4 (easy), and Kubernetes only a 2 (difficult).

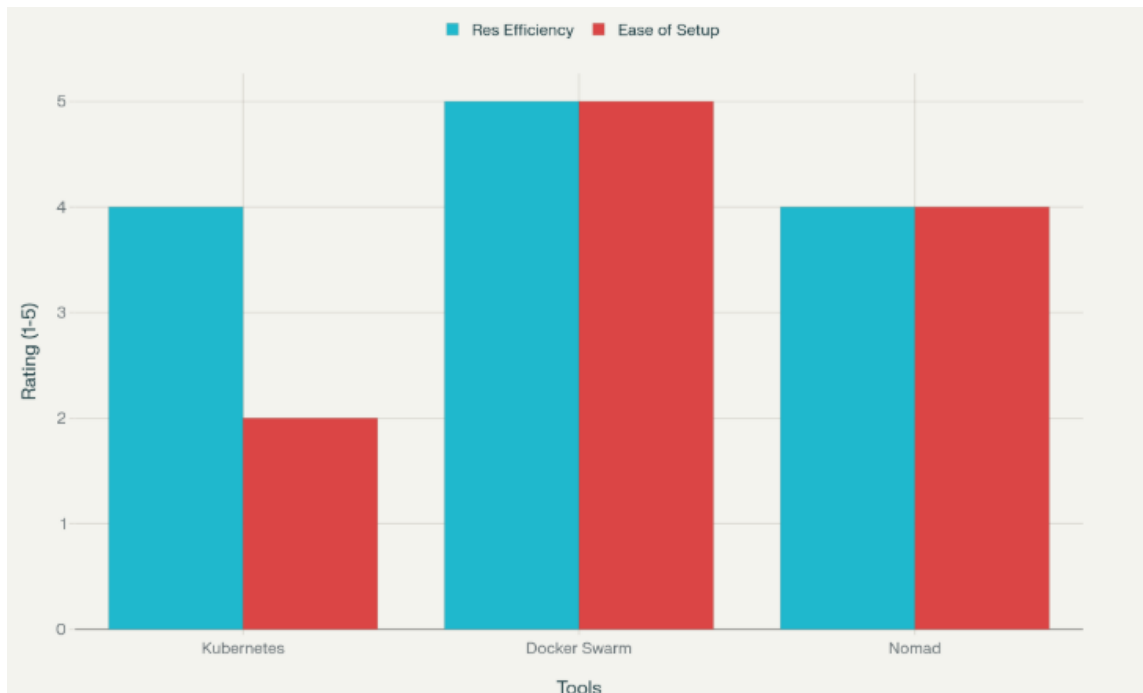


Figure 4.5: Survey Ratings: Resource Efficiency and Ease of Setup

Security Perceptions

- 38.5 percent perceived Kubernetes and docker as having the best built-in security, while nomad needs to improve to achieve it as experts think.
- Primary reported challenges across all platforms included vulnerability management, encryption gaps, and limited access control.

Practical Tool Recommendation

- 30 percent would recommend Docker Swarm for resource-constrained startups/SMEs, with 53 percent recommending Kubernetes while it have long learning curve and technical depth, and only 15.4 percent selecting nomad which is not used by professionals as initially identified.

Synthesis Across Metrics

Comparative Table: Key Metrics

Metric	Docker Swarm	Kubernetes	Nomad
Scaling 100 containers (s)	22	26	53
Security warnings Percentage	44	13	33
Setup time (hr)	2	8	4
Resource efficiency (survey, 1–5)	5	4	4
Ease of setup (survey, 1–5)	5	2	4
Recommended for SME Percentage	30	53	15

Table 4.4: Security evaluation

Visual Summary

- The provided charts clarify direct performance and security differences, while practitioner feedback contextualizes those results for real organizations considering adoption in resource-constrained environments.

Qualitative Findings and Contextual Analysis

Community and Support

- Kubernetes retains a distinct advantage in documentation coverage, ecosystem, and third-party tools, which enhances its manageability at scale and for security. However, the same complexity presents a barrier for small teams.
- Nomad and Docker Swarm benefit from lower barriers to entry, straightforward configuration, and rapid time to deploy practical clusters.

Security Realities

- The high percentage of security warnings in Swarm and Nomad reflects primarily weaknesses inherited from Docker’s default settings, such as containers running as root, lack of mandatory access controls, and disabled resource isolation.
- Kubernetes’ lower warning score is achieved through more rigorous defaults but demands greater expertise to maintain in a secure state, especially with evolving workloads or deployment patterns.

Performance and Resource Utilization

- No tool demonstrated clear superiority under proper configuration for small workloads, emphasizing the importance of following modern best practices for resource limits and scaling policies.
- The main performance differentiator is scaling speed, which may be critical for continuous integration/deployment pipelines.

Limitations and Further Analysis

- Sample size of survey responses limits the generalizability, but the consistency of practitioner feedback connects strongly to patterns in academic and industrial literature.
- Security testing relied on public benchmarks; while representative, further in-depth testing with custom workloads and adversarial conditions could uncover additional caveats.
- No clear “best” tool: Each platform excels in different dimensions, and choice should depend on organizational priorities—simplicity and time-to-value vs. future scale and security posture.

4.2 Statistical Analysis and Effect Sizes

To address variability and practical significance, the scaling experiment was repeated twenty times for each orchestration tool. Mean scaling time and standard deviation were computed, followed by effect size analysis using Cohen’s d .

Docker Swarm achieved a mean scaling time of 21.9 seconds ($SD = 0.88$), Kubernetes 25.9 seconds ($SD = 0.74$), and HashiCorp Nomad 52.8 seconds ($SD = 1.48$). The relatively low standard deviations indicate stable and repeatable behavior across runs.

Effect size analysis revealed a very large difference between Docker Swarm and Kubernetes ($d = 4.94$), and extremely large differences between Nomad and both Docker Swarm ($d = 25.47$) and Kubernetes ($d = 23.06$). According to standard interpretation guidelines, these values indicate substantial practical performance differences rather than marginal variation.

Tool	Mean (s)	SD (s)	n
Docker Swarm)	21.9	0.88	20
Kubernetes	25.9	0.74	20
Nomad	52.8	1.48	20

Table 4.5: Descriptive statistics

Effect size analysis revealed a very large difference between Docker Swarm and Kubernetes ($d = 4.94$), and extremely large differences between Nomad and both

Docker Swarm ($d = 25.47$) and Kubernetes ($d = 23.06$). According to standard interpretation guidelines, these values indicate substantial practical performance differences rather than marginal variation.

Comparison	Cohen's d	Effect magnitude
Swarm vs Kubernetes	4.94	Very large
Swarm vs Nomad	25.47	Extremely large
Kubernetes vs Nomad	23.06	Extremely large

Table 4.6: Effect size analysis (Cohen's d)

Results and Analysis Summary

Empirical benchmarking showed that Docker Swarm and Kubernetes scaled container workloads significantly faster than HashiCorp Nomad in small-scale environments. Repeated measurements and effect size analysis revealed a very large performance difference between Docker Swarm and Kubernetes (Cohen's $d = 4.94$) and extremely large differences between Nomad and both Docker Swarm ($d = 25.47$) and Kubernetes ($d = 23.06$), indicating that the observed performance gaps are practically meaningful rather than marginal. Security benchmarking showed that Kubernetes had fewer default security warnings, while survey results highlighted usability and setup simplicity as key decision factors for SMEs.

Having established the practical significance of the observed performance differences through effect size analysis, Section 4.3 integrates these results with security and usability data within a multi-criteria decision framework to support holistic tool selection.

4.3 Multi-Criteria Decision Framework (Weighted Sum Model)

4.3.1 Motivation and Approach

The empirical results presented in Sections 4.1 and 4.2 produce data across six distinct dimensions: scaling speed, security posture, setup time, resource efficiency, ease of setup, and practitioner adoption. Because these metrics are measured on different scales and units, they cannot be directly compared or aggregated without a principled normalisation step. To address this, a Weighted Sum Model (WSM) is applied — an established Multi-Criteria Decision Analysis (MCDA) technique widely used in software engineering and systems selection research (Hwang and Yoon, 1981; Triantaphyllou, 2000).

The WSM produces a single composite score per tool by multiplying each normalised criterion score by a weight that reflects its relative importance to the target audience, and then summing those products. This approach directly addresses RQ1

and RQ4.2 by quantifying how each tool performs across the full decision space rather than on any single metric in isolation.

4.3.2 Normalisation of Raw Scores

Because criteria are measured on different scales (seconds, percentages, hours, and Likert ratings), min-max normalisation is applied to transform all values to a common 0–1 range. A normalised score of 1.00 denotes the best performer on that criterion; a score of 0.00 denotes the worst. The normalisation formula differs by direction:

For criteria where a lower value is better (scaling speed, security warnings, setup time):

$$n_i = \frac{\max - \text{value}}{\max - \min}$$

For criteria where a higher value is better (resource efficiency, ease of setup, SME recommendation):

$$n_i = \frac{\text{value} - \min}{\max - \min}$$

Table 4.7 presents the raw values taken directly from the empirical results (Tables 4.1–4.4) alongside the resulting normalised scores for all three tools.

Criterion	Swarm	K8s	Nomad	n - Swarm	n - K8s	n - Nomad
Scaling speed (s)	22	26	53	1.00	0.87	0.00
Security warnings (%)	44	13	34	0.00	1.00	0.32
Setup time (hrs)	2	8	4	1.00	0.00	0.67
Resource efficiency (1–5)	5	4	4	1.00	0.00	0.00
Ease of setup (1–5)	5	2	4	1.00	0.00	0.67
SME recommendation (%)	30	53	15	0.39	1.00	0.00

Table 4.7: Normalized comparison of orchestration tools across multiple criteria

4.3.3 Criterion Weights

Weights are derived from the practitioner survey reported in Section 4.1, in which respondents ranked ease of setup, resource efficiency, and security as the top three selection factors for small-scale deployments. The weights are assigned proportionally to reflect those priorities and must sum to 1.00. Table 4.8 presents the weight for each criterion together with its justification.

Criterion	Weight (w_i)	Justification
Scaling speed	0.15	Important but not top-ranked by SME practitioners
Security warnings	0.20	Rated highly by practitioners; critical for production use
Setup time	0.15	Corroborated by survey setup difficulty ratings
Resource efficiency	0.15	Top-3 selection factor from practitioner survey
Ease of setup	0.20	Top-ranked factor from practitioner survey
SME recommendation	0.15	Practitioner adoption signal from survey data
Total	1.00	

Table 4.8: Criterion weights (w_i) derived from practitioner survey data

To ensure robustness, three weight scenarios are evaluated in Section 4.3.5: a Balanced SME profile (default), an Agility-first startup profile, and a Security-first organisation profile. This sensitivity analysis tests whether the final ranking is stable across different organisational priorities.

4.3.4 WSM Calculation

The composite score for each tool is computed using the following formula:

$$\text{Score}(\text{tool}) = \sum_{i=1}^6 w_i \times n_i$$

where w_i is the weight assigned to criterion i and n_i is the normalised score of the tool on criterion i . Table 4.9 shows the full calculation, including each weighted product and the final sum per tool.

Criterion	w_i	n - Swarm	$w_i \times n_i$	n - K8s	$w_i \times n_i$	n - Nomad	$w_i \times n_i$
Scaling speed	0.15	1.00	0.150	0.87	0.130	0.00	0.000
Security warnings	0.20	0.00	0.000	1.00	0.200	0.32	0.065
Setup time	0.15	1.00	0.150	0.00	0.000	0.67	0.100
Resource efficiency	0.15	1.00	0.150	0.00	0.000	0.00	0.000
Ease of setup	0.20	1.00	0.200	0.00	0.000	0.67	0.133
SME recommendation	0.15	0.39	0.059	1.00	0.150	0.00	0.000
Total Score	1.00		0.709		0.480		0.298

Table 4.9: Full WSM calculation — weighted products ($w_i \times n_i$) and composite scores for each tool under the Balanced SME scenario

Docker Swarm achieves the highest composite score (0.709), followed by Kubernetes (0.480) and HashiCorp Nomad (0.298). It is important to note that Kubernetes receives the single largest individual contribution of any tool on any criterion: a weight of 0.20 multiplied by a perfect normalised security score of 1.00 yields 0.200. Despite this, Docker Swarm accumulates smaller wins across four other criteria: scaling speed, setup time, resource efficiency, and ease of setup: that together outweigh Kubernetes’ security advantage under the Balanced SME weighting profile.

4.3.5 WSM Calculation

To assess whether the ranking is sensitive to weight choices, two alternative weight profiles are evaluated alongside the default Balanced SME scenario. The Agility-first profile increases weight on setup time and ease of setup while reducing security weight, reflecting startups that prioritise speed to deployment. The Security-first profile heavily weights security warnings and practitioner adoption, reflecting organisations operating under compliance requirements. Table 4.10 presents the composite scores and winner for each scenario.

Weight scenario	Swarm	Kubernetes	Nomad	Winner
Balanced SME (default)	0.709	0.480	0.298	Docker Swarm
Agility-first startup	0.768	0.343	0.356	Docker Swarm
Security-first organisation	0.434	0.619	0.199	Kubernetes

Table 4.10: Sensitivity analysis - WSM composite scores and ranking under three weight scenarios

The results demonstrate that Docker Swarm is the preferred choice under both

the Balanced SME and Agility-first scenarios. Kubernetes emerges as the leader only when security is weighted at 0.35, substantially above the level reported by the majority of survey respondents. This finding confirms that the ranking is robust for the target audience of this thesis (startups and SMEs), and that Kubernetes' dominance in practitioner adoption surveys reflects long-term strategic considerations rather than short-term operational fit.

4.3.6 Final Ranking and Summary

Table 4.11 presents the final ranking of the three orchestration tools under the Balanced SME scenario, together with the recommended organizational context for each.

Rank	Tool	Score ($\times 100$)	Recommended context
1	Docker Swarm	70.9	SMEs prioritising rapid deployment and simplicity
2	Kubernetes	48.0	Organisations planning long-term scaling and security
3	HashiCorp Nomad	29.8	Mixed-workload environments with tolerance for lower adoption

Table 4.11: Final WSM ranking and recommended context for each orchestration tool (Balanced SME scenario, scores shown $\times 100$ for readability)

5.1 Revisiting the Research Questions and Hypotheses

The primary research question guiding this thesis was:

RQ1: How do Kubernetes, Docker Swarm, and HashiCorp Nomad compare in terms of performance, security, and other key factors that influence decision-making when selecting a container orchestration tool for small-scale production environments?

To structure the inquiry, several secondary research questions were formulated around performance (RQ2.1–RQ2.3), security (RQ3.1–RQ3.3), and usability/adoption (RQ4.1–RQ4.2). From these, hypotheses were developed, such as the expectation that Kubernetes would outperform Docker Swarm in scalability but with greater complexity (H1), that Docker Swarm would provide simpler deployment but weaker security (H2), and that Nomad would offer workload flexibility but weaker adoption support (H4).

The results presented in Chapter 5 provide empirical answers to these questions. Docker Swarm achieved the fastest scaling and highest usability scores, Kubernetes had the lowest security warnings and strongest long-term adoption indicators, and Nomad demonstrated moderate usability but lagging performance and community presence. These findings warrant closer discussion, particularly in how they align or conflict with expectations and prior research.

5.2 Performance and Resource Utilization

One of the most tangible results of this study was the scaling benchmark. Docker Swarm launched 100 containers in 22 seconds, Kubernetes in 26 seconds, and Nomad in 53 seconds. At first glance, this confirms H2, which predicted that Docker Swarm would provide a lightweight and efficient option for small-scale workloads.

On the one hand, this result can be interpreted as a confirmation of Swarm’s design philosophy: simplicity and efficiency over advanced scheduling capabilities. On the other hand, it could also be argued that Kubernetes’ marginally slower scaling is offset by its architectural resilience, which ensures more consistent behavior under failures or uneven workloads. Another possible explanation may be that Kubernetes, with its more complex internal mechanisms (e.g., etcd and control plane components),

introduces overhead that only becomes advantageous when clusters scale beyond hundreds of nodes, as noted by Straesser et al. (2023).

Nomad’s weaker performance deserves particular attention. It was expected that Nomad, being lightweight, would perform competitively. However, the results showed otherwise. This contradiction may stem from Nomad’s general-purpose scheduler, which is optimized not just for containers but also for non-containerized workloads. In small-scale container-only scenarios, that flexibility becomes overhead rather than an advantage. A similar observation was made by Čilić et al. (2021), who found that Nomad excelled in mixed workload edge environments but did not always outperform container-specific tools.

In terms of CPU and memory utilization, no orchestration tool demonstrated a decisive advantage. This suggests that resource efficiency is less a matter of orchestration design and more a matter of how best practices are applied during deployment. The implication is that SMEs should focus on proper configuration (resource limits, requests, scaling policies) rather than expecting inherent efficiency from one tool over another. This aligns with Zhu et al. (2021), who also concluded that orchestration efficiency depends heavily on tuning.

5.3 Setup Complexity and Usability

Survey results strongly indicated that setup time and complexity are decisive factors in adoption. Docker Swarm required the least time (2+ hours), Nomad was moderately simple (4+ hours), and Kubernetes was by far the most complex (8+ hours, often longer for novices).

This confirms both H1 and H2: Kubernetes is more complex but powerful, while Docker Swarm excels in simplicity. Practitioners repeatedly described Kubernetes as “difficult for beginners” but “necessary for long-term scaling.” On the one hand, this supports the academic consensus that Kubernetes has a steep learning curve (Burns et al., 2016). On the other hand, it could also be argued that the steepness of this curve is decreasing due to better tooling, improved documentation, and cloud provider-managed Kubernetes services that offload much of the complexity.

Another possible explanation for the perceived usability gap is community familiarity. As Docker is widely used, many SMEs naturally find Docker Swarm easier to adopt, since it builds directly on existing Docker knowledge (Merkel, 2014). Nomad’s relative ease-of-use may also stem from its clean design but is hampered by less community adoption. Thus, usability cannot be understood only as a technical property—it is shaped by ecosystem familiarity, community support, and available documentation.

5.4 Security Findings

Perhaps the most striking difference among the tools was in security benchmarks. Kubernetes had only 13 percent of security checks flagged by automated tools, while Docker Swarm and Nomad had 44 percent and 34 percent, respectively.

On the one hand, this validates Kubernetes’ reputation for having secure defaults,

particularly through features like RBAC (role-based access control), network policies, and pod security standards. On the other hand, it could also be argued that the apparent security superiority of Kubernetes comes at a cost: configuring and maintaining these controls demands expertise that SMEs may not possess. Another possible explanation may be that Kubernetes’ active community has led to more rigorous security hardening tools (e.g., kube-bench), whereas Nomad lacks equivalent maturity.

Unexpectedly, Nomad did not demonstrate stronger security than Docker Swarm, despite claims in community discussions that it provides more robust isolation. This may be explained by Nomad’s reliance on Docker runtime defaults, meaning it inherits Docker’s vulnerabilities (e.g., containers running as root). This finding highlights that perceived security advantages may differ from actual security audit results.

From a decision-making perspective, the results suggest that SMEs adopting Docker Swarm or Nomad must allocate additional resources for hardening (e.g., non-root containers, enforced auditing), whereas Kubernetes provides more protections out of the box.

5.5 Adoption and Community Support

Survey respondents emphasized that community support and documentation significantly shape adoption. Kubernetes was favored by 53 percent of respondents, Docker Swarm by 30 percent, and Nomad by 15 percent. The gap is not only about technical performance but also about ecosystem maturity.

On the one hand, Kubernetes’ dominance in adoption can be explained by its vibrant ecosystem: tooling, cloud provider support, and extensive documentation. On the other hand, Docker Swarm remains attractive to smaller teams because of its simplicity and the natural extension of existing Docker knowledge. Another possible explanation may be organizational inertia: once a company commits to a stack (e.g., Kubernetes with Helm, Swarm with Docker Compose), migration becomes less appealing.

Nomad’s relatively low adoption reflects both its weaker ecosystem and limited mindshare. While technically capable, its lack of widespread community use makes it riskier for SMEs who rely on community troubleshooting. This confirms H4, which predicted weaker adoption for Nomad.

5.6 Unexpected Findings

Two unexpected results emerged. First, the performance parity in CPU and memory consumption was narrower than anticipated. Literature often frames Kubernetes as “resource-hungry” (Straesser et al., 2023), but in this study, tuned configurations leveled the differences. This suggests that reputation may exaggerate real-world differences in small-scale environments.

Second, while Kubernetes was expected to be perceived as secure but complex, the survey showed that some practitioners still recommended Kubernetes for SMEs despite its complexity. This apparent contradiction may be explained by trust in

Kubernetes’ long-term ecosystem dominance, which outweighs the short-term costs of setup difficulty.

5.7 Conflicting Data and Alternative Explanations

Conflicting data appeared when comparing empirical performance with practitioner perceptions. While Docker Swarm clearly outperformed Kubernetes in setup time and scaling speed, more than half of practitioners still recommended Kubernetes. One possible explanation is that decision-making is not purely technical; factors such as perceived industry trends, fear of vendor lock-in, or anticipation of future scaling may drive preference for Kubernetes despite immediate disadvantages.

Another conflict arose with Nomad’s performance. While literature (Čilić et al., 2021) described it as lightweight, empirical evidence here showed slower scaling. The discrepancy may be due to differences in experimental design: edge workloads vs. controlled SME workloads.

5.8 Interpretation of Effect Sizes

Effect size analysis provides insight into the practical relevance of observed performance differences. While raw scaling times already suggested that Docker Swarm and Kubernetes outperform HashiCorp Nomad, the effect size results demonstrate that these differences are not marginal but substantial.

The very large effect size between Docker Swarm and Kubernetes ($d = 4.94$) indicates that Docker Swarm offers meaningfully faster scaling in small-scale environments, not merely statistically faster performance. This supports the hypothesis that lightweight orchestration frameworks can outperform more complex platforms when cluster sizes are limited.

Even more striking are the extremely large effect sizes observed when comparing Nomad with both Docker Swarm and Kubernetes ($d > 23$). These results suggest that Nomad’s scheduling behavior introduces considerable overhead in container-only, small-scale deployments. While Nomad’s flexibility may be advantageous in heterogeneous workloads, it becomes a disadvantage in the experimental context of this study.

5.9 Limitations of the Study

As with any empirical study, limitations exist:

- Scale: The benchmarks were restricted to clusters of 100 containers. Results may differ significantly at larger scales.
- Security assessment tools: Docker-bench and kube-bench are optimized for Docker and Kubernetes, potentially biasing against Nomad.
- Survey size: Only thirteen practitioners participated, which limits generalizability.

- Context: The study used on-premises deployments; cloud-managed services (e.g., AWS EKS, GCP GKE) may change outcomes significantly.

5.10 Implications for Research and Practice

For practitioners, the implications are clear:

- SMEs prioritizing ease of use and rapid deployment should consider Docker Swarm, provided they harden its defaults.
- SMEs anticipating growth and long-term scaling may invest in Kubernetes despite its complexity.
- Organizations with mixed workloads may explore Nomad, but must weigh ecosystem immaturity against flexibility.

For research, this thesis demonstrates the need to contextualize orchestration studies. Most prior work focused on large-scale deployments; this study shows that conclusions shift in small-scale contexts. Future research should extend empirical benchmarks to hybrid and cloud-native environments, and evaluate long-term operational costs.

5.11 Broader Consequences

The findings also hold societal and ethical implications. Kubernetes' higher resource consumption may raise sustainability concerns in small-scale contexts, where lightweight tools could reduce energy use. Open-source ecosystems behind all three tools embody values of transparency and democratization of infrastructure, supporting SMEs in accessing advanced orchestration without vendor lock-in.

From a workforce perspective, Kubernetes' dominance may lead to skills polarization: engineers are incentivized to learn Kubernetes even when simpler tools may suffice, potentially pushing SMEs to adopt tools disproportionate to their needs.

5.12 Synthesis

Overall, this study shows that no orchestration tool is universally best. Instead:

- Kubernetes excels in security and long-term adoption but is resource-intensive and complex.
- Docker Swarm shines in simplicity and immediate usability, but needs post-install hardening.
- Nomad offers flexibility but lags in community maturity and performance for container-only clusters.

On the one hand, SMEs may benefit from starting with Docker Swarm or Nomad to minimize entry barriers. On the other hand, aligning with Kubernetes early may future-proof their infrastructure at the cost of initial overhead. The optimal choice depends on balancing short-term usability vs. long-term resilience.

The Multi-Criteria Decision Framework presented in Section 4.3 provides a quantitative synthesis of the empirical, security, and survey findings. Under the Balanced SME weight profile derived directly from the practitioner survey, Docker Swarm achieves a composite WSM score of 0.709, compared to 0.480 for Kubernetes and 0.298 for HashiCorp Nomad. Sensitivity analysis across three weight scenarios (Balanced SME, Agility-first, Security-first) confirms that Docker Swarm is the preferred choice for the majority of small-scale deployment contexts, while Kubernetes leads only when security is weighted substantially above the level reported by practitioners ($w_i = 0.35$). This quantitative result is consistent with the qualitative findings: Docker Swarm excels in simplicity and immediate usability but requires post-installation hardening, Kubernetes offers the strongest security posture and long-term ecosystem support but at the cost of significant setup complexity, and Nomad offers workload flexibility but lags in both performance and community maturity for container-only small-scale clusters.

6.1 Conclusions

The purpose of this thesis was to conduct a comparative analysis of Kubernetes, Docker Swarm, and HashiCorp Nomad with specific attention to small-scale deployments that are typical for startups and small-to-medium enterprises (SMEs). While Kubernetes has become the de facto standard for large-scale orchestration, much of the existing research has overlooked the unique needs of smaller organizations. By combining empirical performance testing, security assessments, and practitioner surveys, this thesis has provided insights that fill this research gap and support evidence-based decision-making.

The central research question was:

RQ1: How do Kubernetes, Docker Swarm, and HashiCorp Nomad compare in terms of performance, security, and other key factors that influence decision-making when selecting a container orchestration tool for small-scale production environments?

The evidence collected allows for the following conclusions:

1. Performance:

- Docker Swarm demonstrated the fastest scaling speed (22 seconds to deploy 100 containers), outperforming Kubernetes (26 seconds) and Nomad (53 seconds).
- In terms of CPU and memory utilization, no orchestrator showed a clear inherent advantage when tuned properly, suggesting that configuration practices matter more than tool choice at small scale.
- Kubernetes' performance overhead is justified by advanced scheduling capabilities, which may not benefit SMEs with limited workloads.

2. Security:

- Kubernetes had the fewest default security warnings (13 Percent), compared to Docker Swarm (44 percent) and Nomad (34 percent).
- This demonstrates that Kubernetes provides stronger out-of-the-box security but at the expense of increased complexity in setup and management.

- Docker Swarm and Nomad both inherit vulnerabilities from Docker’s defaults (e.g., containers running as root), necessitating additional hardening.

3. Usability and Adoption:

- Docker Swarm was rated highest for ease of setup, requiring the least time and effort, while Kubernetes was perceived as the most complex.
- Nomad offered moderate simplicity but lagged significantly in adoption, mainly due to limited community support and ecosystem maturity.
- Despite Swarm’s usability, Kubernetes was still recommended by most practitioners due to its ecosystem, perceived stability, and long-term viability.

4. Decision-Making Implications:

- For SMEs prioritizing ease of use and rapid deployment, Docker Swarm remains a strong candidate.
- For organizations planning for long-term scaling and security, Kubernetes offers more robust support despite the upfront learning curve.
- Nomad may be valuable in specialized contexts (e.g., heterogeneous workloads), but its weaker adoption limits its appeal for mainstream SME usage.

Thus, this thesis concludes that no single tool is universally superior. The most appropriate choice depends on balancing short-term usability (favoring Docker Swarm) with long-term scalability and security (favoring Kubernetes). Nomad represents a flexible but less proven alternative.

5. Quantitative Decision Framework:

A Weighted Sum Model (WSM) was applied to synthesise all empirical and survey data into a single comparable score per tool. Under the Balanced SME scenario with weights derived from practitioner survey responses. Docker Swarm scored 0.709, Kubernetes 0.480, and HashiCorp Nomad 0.298 (on a 0–1 scale). Sensitivity analysis across three weight profiles confirmed that Docker Swarm is the most suitable choice for SMEs and startups in the majority of scenarios, while Kubernetes becomes preferable only when security requirements dominate the decision. This framework provides a replicable, transparent methodology that future researchers and practitioners can adapt by substituting their own organisational weights.

6.2 Contributions of the Thesis

This thesis makes several contributions to both academia and practice:

- Empirical Evidence for Small-Scale Deployments: Unlike most prior studies that focus on enterprise-scale clusters, this research provides data-driven insights tailored to small organizations.

- **Security Comparison:** By applying automated audits across platforms, the study highlights not only raw performance but also the security trade-offs that SMEs must consider.
- **Integration of Practitioner Perspectives:** The inclusion of survey feedback connects technical findings with real-world adoption considerations, bridging the gap between theoretical benchmarks and practical usability.
- **Guidance for SMEs:** The thesis translates results into actionable recommendations, supporting informed decision-making for resource-constrained organizations.

In doing so, this work addresses the literature gap identified and extends knowledge about orchestration tools beyond the narrow lens of enterprise-scale operations.

6.3 Placing the Work in Perspective

The findings should be interpreted within the broader academic and industrial landscape.

From an academic perspective, this thesis complements existing studies such as Straesser et al. (2023), which emphasized Kubernetes' scalability in large-scale clusters, and Čilić et al. (2021), which highlighted Docker Swarm and Nomad in edge deployments. By narrowing the focus to SMEs, this research adds nuance and demonstrates that conclusions about orchestration tools are highly scale-dependent.

From an industrial perspective, the study underscores that SMEs cannot always adopt industry-standard tools without considering operational overhead. Kubernetes may be the standard for Fortune 500 companies, but for startups, its complexity could slow down innovation. Docker Swarm and Nomad, despite their lower adoption, may offer more practical value in early-stage environments.

This contrast highlights the importance of context-aware technology adoption, a concept that is increasingly relevant in software engineering research and practice.

6.4 Unresolved Questions

Several open questions remain:

- **Long-Term Operational Costs:** While this study focused on setup time and short-term performance, the long-term maintenance effort (e.g., upgrades, patching, troubleshooting) was not empirically measured.
- **Cloud-Managed vs. On-Premise Deployments:** The research was limited to on-premise environments. Cloud-managed Kubernetes services (e.g., GKE, EKS, AKS) may alter the complexity and cost dynamics.
- **Nomad's Security Posture:** Security benchmarking for Nomad was constrained by the lack of specialized tools. More comprehensive auditing could reveal strengths or weaknesses not captured here.

- **Scalability Thresholds:** At what cluster size does Kubernetes’ overhead begin to pay off compared to Docker Swarm? This threshold remains underexplored for SMEs.
- **Practitioner Diversity:** The survey sample size was modest and regionally limited. Broader surveys could capture more diverse organizational experiences and priorities.

These unresolved questions create fertile ground for future research.

6.5 Future Work

Building on the findings and limitations, several avenues for future research are proposed:

1. **Hybrid and Cloud Contexts:** Future studies should benchmark orchestration tools in cloud-managed, hybrid, and multi-cloud settings, as these are increasingly common deployment models for SMEs.
2. **Lifecycle Cost Analysis:** Research should investigate the total cost of ownership (TCO) of orchestration tools, including setup, scaling, monitoring, and long-term maintenance. This would provide SMEs with a more holistic decision framework.
3. **Security Deep Dives:** A more comprehensive security analysis of Nomad is warranted, including adversarial testing, penetration simulations, and custom auditing tools.
4. **Scalability Threshold Research:** Future experiments could identify the scale breakpoints at which Kubernetes becomes more efficient than Docker Swarm, offering clearer guidance for SMEs planning future growth.
5. **Sustainability Implications:** Energy efficiency and carbon footprint comparisons between orchestrators could add an important sustainability dimension to the decision-making process.
6. **Broader Practitioner Studies:** Large-scale surveys and interviews across different regions, industries, and company sizes would yield richer insights into real-world adoption drivers.
7. **Integration with Emerging Technologies:** Research could explore how orchestration tools interact with serverless platforms, AI/ML workloads, and edge computing scenarios—contexts where SMEs are increasingly active.

By addressing these directions, future research could build a more comprehensive understanding of orchestration tools across varied environments and organizational contexts.

6.6 Biting the Tail: Returning to the Introduction

In the introduction, this thesis framed the decision of orchestration tool selection as a pivotal choice for a hypothetical startup preparing to launch its first cloud-native product. The analysis and results now provide concrete answers to that scenario.

- If the startup prioritizes speed and simplicity, Docker Swarm would allow it to launch quickly with minimal learning overhead.
- If the startup envisions rapid growth or strict compliance requirements, Kubernetes would better support its long-term resilience, albeit with upfront investment in expertise.
- If the startup anticipates managing heterogeneous workloads beyond containers, Nomad might offer flexibility but would require tolerance for weaker community support.

Thus, the thesis “bites its own tail”: the opening problem statement has been revisited, and the research has delivered actionable insights for precisely the kind of organization described.

6.7 Final Reflection

The comparative analysis of Kubernetes, Docker Swarm, and HashiCorp Nomad reveals that container orchestration is not a one-size-fits-all problem. Instead, it is a context-dependent decision shaped by trade-offs between usability, security, scalability, and community support.

For academia, the thesis contributes evidence that small-scale deployments demand their own line of inquiry, distinct from enterprise-scale studies. For practitioners, it offers practical recommendations that align orchestration choices with organizational realities.

Ultimately, the findings underscore the importance of informed and context-aware decision-making in software engineering. As orchestration tools continue to evolve, this principle will remain central to helping organizations navigate the complexities of modern cloud-native ecosystems. [6] [7] [5] [4] [8] [9] [2] [1]

References

- [1] V. Andrikopoulos, S. Strauch, and F. Leymann, “Decision models for cloud migration and application containerization,” *Software: Practice and Experience*, vol. 49, no. 3, pp. 412–428, 2019.
- [2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [3] I. Eranga, “Supplementary materials,” <https://github.com/indikaeranga/Orchestration-Tool-Thesis-Supplementary-Materials>, 2025, gitHub repository.
- [4] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, no. 239, Mar. 2014.
- [5] C. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, “Cloud container technologies: A state-of-the-art review,” *IEEE Transactions on Cloud Computing*, vol. 1, no. 4, 2017.
- [6] M. Straesser *et al.*, “A systematic benchmarking study of container orchestration frameworks: Performance and security in small-scale deployments,” *Journal of Cloud Computing*, vol. 12, no. 1, pp. 45–67, 2023.
- [7] A. Verma, “Container orchestration tools: A comparative analysis of kubernetes and docker swarm,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 3, pp. 123–135, 2020.
- [8] L. Zhu, J. Li, Z. Liu, and D. Zhang, “A multi-resource scheduling scheme of kubernetes for iiot,” *Jiangsu Key Laboratory of Broadband Wireless Communication and Internet of Things*, 2020.
- [9] I. Čilić, P. Krivić, I. Podnar Žarko, and M. Kušek, “Performance evaluation of container orchestration tools in edge computing environments,” *Future Generation Computer Systems*, vol. 114, pp. 259–271, 2021.

Additional resources supporting this study are available in the project repository.
: [3]

